

DeLTA 3.0: A modern segmentation and tracking tool for microscopy images

Virgile Andreani^{1,2}, Jean-Baptiste Lugagne^{1,3}, Owen M. O'Connor¹, and Mary J. Dunlop¹

¹ Biomedical Engineering, Boston University, USA ² EPI Lifeware, Inria Saclay, France ³ Department of Engineering Science, University of Oxford, United Kingdom ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [David Stansby](#)

Reviewers:

- [@msschwartz21](#)
- [@JojoDevel](#)

Submitted: 10 October 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Time-lapse microscopy is a popular and widely used approach for quantifying single-cell dynamics. Analysis of sequences of images requires two steps: segmentation—locating the cells in each frame—and tracking—identifying how cells in one frame relate to the next. DeLTA (Deep Learning for Time-lapse Analysis) is a Python ([Perez et al., 2011](#)) package that performs segmentation and tracking. It was initially developed to handle images of bacteria growing within the “mother machine,” ([Wang et al., 2010](#)) a popular microfluidic device for single-cell analysis that constrains cells to grow in one dimension ([Lugagne et al., 2020](#)), and was later extended to handle cells growing in two dimensions ([O'Connor et al., 2022](#)). After three years of experience and feedback from our group and the broader community, we are releasing version 3.0—a major overhaul designed to achieve four key objectives: enhance usability for scientists, improve modularity and adaptability for developers, integrate common features like growth rate computation and lineage manipulation, and adopt modern open-source development practices.

Statement of need

DeLTA is a deep learning pipeline for analysing bacterial time-lapse microscopy images.

This third version adds functionalities while remaining true to DeLTA's principles. DeLTA distinguishes itself from other packages by:

- the simplicity of its implementation, making its code pedagogical and easily adaptable;
- its built-in handling of mother machine settings, a widely-used device where segmentation and tracking are performed within several regions of interest on each frame;
- its ability to cater to novice and experienced users alike, respectively through a new interface allowing users to run and train DeLTA without writing code, and through an API allowing programmatic and real-time use;
- new utilities provided for common analysis functions (e.g., robust estimation of growth rate, correcting tracking errors).

In addition, in this version, deep learning models have been ported to keras ([Chollet & Others, 2015](#)), a high-level backend-agnostic library that allows users to select any of the three main deep learning backends (tensorflow ([Abadi et al., 2015](#)), torch ([Ansel et al., 2024](#)), or jax ([Bradbury et al., 2018](#))).

State of the field

When DeLTA was originally developed, only CellProfiler ([Stirling et al., 2021](#)) and DeepCell ([Valen et al., 2016](#)) existed in the Python ecosystem for cell segmentation and tracking

39 using deep learning. Since then, several other tools have been developed, including StarDist
40 (Schmidt et al., 2018; Weigert et al., 2020) (limited to the segmentation of star-convex shapes:
41 not appropriate for filamenting cells), Cellpose (Stringer et al., 2021), Omnipose (Cutler et
42 al., 2022), MM3 (Thiermann et al., 2024), TrackAstra (Gallusser & Weigert, 2025), and Cell-
43 TRACTR (O'Connor & Dunlop, 2025). DeLTA's first version was developed around the mother
44 machine. To date, only DeLTA and MM3 offer built-in mother machine support. The second
45 version of DeLTA extended analysis to two-dimensional growth, further expanding the utility
46 of the software. The current version makes it to our knowledge the only segmentation and
47 tracking package to also to provide robust growth rate estimation and utilities to manipulate
48 lineages and fix tracking errors.

49 Software Design

50 DeLTA's design philosophy is based on a trade-off around four principles: (1) user-friendly
51 defaults and interfaces to empower less-technical users, (2) a modular API to allow expert
52 users to address all situations, (3) minimizing custom code when reliable and performant
53 libraries exist, and (4) providing reliable utilities for the most common needs of users beyond
54 pure segmentation and tracking.

55 Version 3 aims to improve all four principles:

- 56 1. The command-line interface that this version introduces enables the use of DeLTA
57 without writing code for segmentation, tracking, and model training.
- 58 2. The API has been rationalized to reduce naming and usage inconsistencies.
- 59 3. We adopted an open data model, based on xarray (in memory) and netCDF4 (for file
60 storage), to replace our custom pickle-based object serialization, to allow any third-party
61 package to open DeLTA result files. Details and trade-offs related to this approach are
62 described in section 9.
- 63 4. We provided two utilities to address exceedingly common needs of experimentalists: the
64 computation of instantaneous cell growth rate, and an interface to fix tracking errors.
65 See section 8 and section 7 for details on the implementations of these features.

66 Research Impact Statement

67 As one of the first packages using deep learning for the segmentation and tracking of single
68 cells, DeLTA has had a significant research impact (Ahmadi et al., 2024; Allard et al., 2022;
69 Gericke et al., 2025) and is an inspiration or comparison point for the development of other
70 packages (O'Connor & Dunlop, 2025; Thiermann et al., 2024).

71 DeLTA is widely used by researchers internationally. The articles describing earlier versions
72 have accumulated a total of >350 citations to date.

73 Currently, DeLTA is downloaded ~220 times per month on PyPI and ~430 times per month on
74 conda-forge.

75 General principles of the package

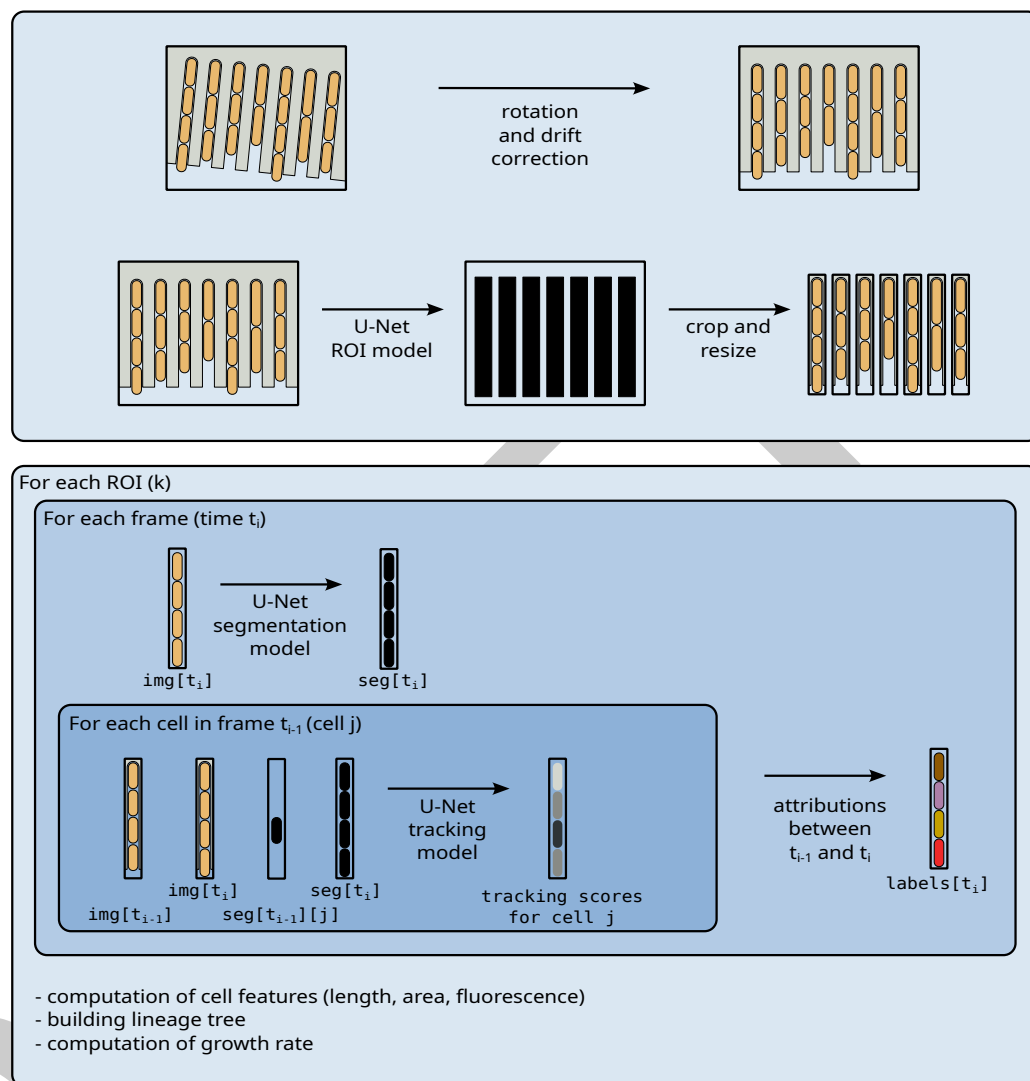


Figure 1: Schematic principles of DeLTA. First, any rotation and drift are removed from the movie. Then, in the case of a mother machine experiment, a region of interest (ROI) model detects and crops individual chambers. Then, for each ROI, each frame is segmented with a U-Net, and passed to a tracking model with the previous frame and the mask of a single cell in the previous frame, to generate tracking scores that determine attribution. Finally, morphological cell features are computed, the lineage tree is built, and growth rates are computed.

76 DeLTA uses three different U-Net (Ronneberger et al., 2015) models: for region of interest
77 (ROI) identification, segmentation, and tracking. Once ROIs have been identified and resized,
78 the segmentation model is run frame-by-frame to determine the location of cells. Finally, for
79 every consecutive pair of frames, the tracking model is run once per cell in the earlier frame,
80 to determine the probabilities that this cell corresponds to each cell in the later frame. After
81 mother-daughter relationships have been determined, a lineage tree is constructed [section 7]
82 where cell features are stored: area, length, average fluorescence, and growth rate [section 8].
83 Finally, these results are saved as a netCDF file accompanied by a segmented movie for direct
84 visualization.

Lineage

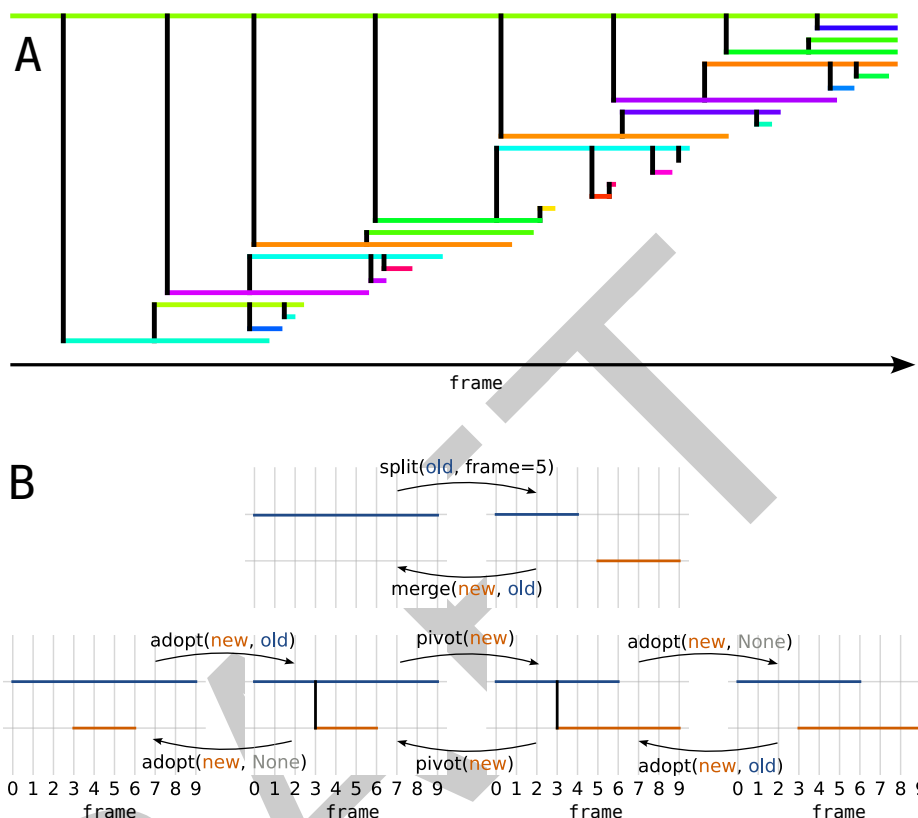


Figure 2: Lineage tree manipulation. **A:** Lineage tree of one chamber from a mother machine experiment. Horizontal lines represent individual cells, while vertical black lines represent division events. Cells exit at the bottom of the mother machine, so some lineages terminate before the end of the figure. The top “mother” cell is trapped and remains in the mother machine throughout the full time range. **B:** This set of functions can be used to correct any tracking errors. If DeLTA fails to associate one cell with itself in the next frame, the merge function can be used to mend the line. The split function does the opposite. The adopt function can set or unset a mother-daughter relationship, while pivot switches a mother and a daughter (the mother being by convention the cell with the oldest pole).

The tracking functionality of DeLTA detects cell divisions, linking mother cells to their daughters. Bacterial cell divisions are typically symmetrical, however pole age has measurable effects on cell physiology, where the daughter cell that inherits the older end of the mother cell grows more slowly than the other daughter (Stewart et al., 2005). For this reason, we consider that the mother cell persists through divisions, giving birth to one daughter (instead of the mother cell disappearing and giving rise to two daughter cells). Thus, upon cell division, the mother cell retains its ID number, while the new cell is given a new ID.

DeLTA constructs the lineage tree frame-by-frame, by extending existing cells and creating new daughter cells at divisions. However, tracking errors are possible, for example due to a temporary air bubble in the field of view, or by a segmentation or a tracking mistake by the neural networks. These cases can be corrected by relabeling the cells involved. However, relabeling requires updating an entire lineage subtree, an error-prone operation. For this reason, we implemented a set of elementary operations on the tree, whose combined effect allows for arbitrarily complex lineage tree manipulations [Figure 2].

Growth rate

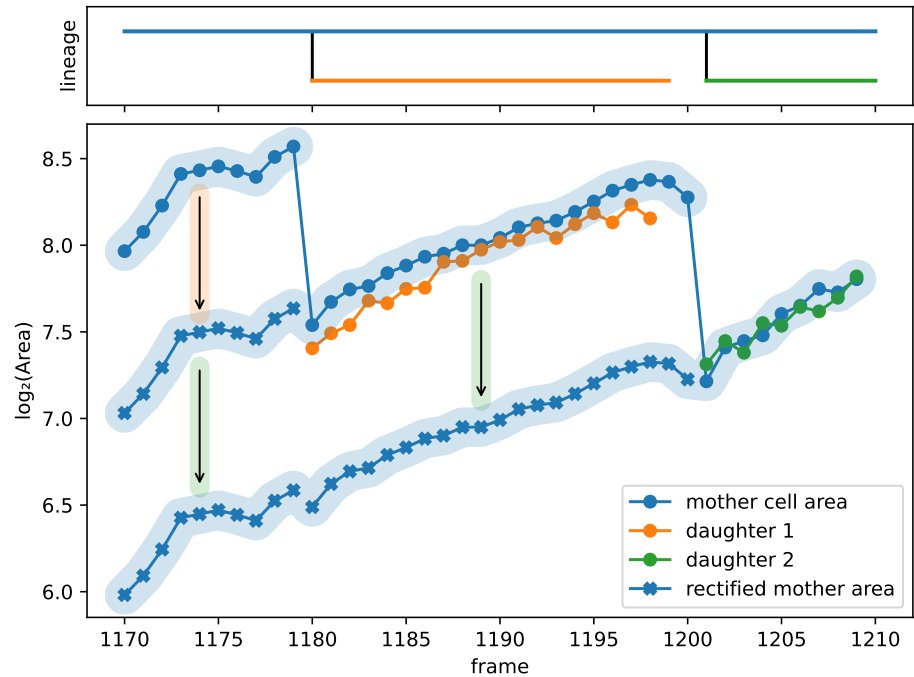


Figure 3: Alignment of cell areas for growth rate computation. Circle markers show the logarithm of the area (number of pixels) of a mother cell and two of its daughters. Jumps are where the cell divides. Cross markers show entire cell cycles that are offset (colored arrows) as a function of the relative sizes of the cells after division (highlighted pair of points). The offset is $\log_2 \left(1 + \frac{\text{Area}_{\text{mother}}}{\text{Area}_{\text{daughter}}} \right)$. After offsetting, the growth rate can be computed at any point, even with a centered difference scheme.

Single-cell growth rate is often used as a proxy for cell fitness. Within a given population, individual cells have different growth rates, which can vary with time. The single-cell growth rate, $\mu(t)$, satisfies the following differential equation with $\ell(t)$ the cell length:

$$\frac{d\ell}{dt} = \mu(t)\ell(t)$$

DeLTA can calculate growth rates based on cell area or length. Area-based growth rates rely on direct counts of the number of pixels per cell, while length-based growth rates derive length estimates from the bacterial shape. Growth rates based on volume are not possible to calculate with images at a single focal plane.

Instantaneous growth rate is computed in DeLTA as the derivative of the logarithm of cell length (or area). Because segmentation can make this time series noisy, we use a Savitzky-Golay filter (Savitzky & Golay, 1964) to simultaneously smooth and differentiate the signal. It is a centered finite difference scheme, which is a deliberate choice to avoid discretization biases.

To compute the growth rate, the Savitzky-Golay filter requires at least one data point from the previous frame and one from the next (or more than one for larger window sizes). To avoid apparent jumps in growth rate around cell division events, we offset the areas before and after division by extrapolation of the lineage. This amounts to considering that the area of a mother cell after division is the sum of its own area and that of its daughter [Figure 3].

File format of results

Analysis results produce image-like data (segmentation masks and label masks), as well as scalar data associated with every cell (ID of mother cell), or with every cell and frame (features such as length, area, average fluorescence, growth rate). In DeLTA 2.0 we used the pickle library to serialize the Python object representing the results into a file. While this allowed for saving arbitrary Python objects, these files could only be opened with the same version of DeLTA that created them.

In version 3 we now use netCDF4 (Rew et al., 1989), an open file format based on HDF5 (The HDF Group, 2025), popular for storing scientific data with libraries available in many programming languages. A limitation of this approach, compared to pickle files, is that the HDF5 data format only supports collections of rectangular data arrays, which required us to reorganize the data describing cell features and lineages in a rectangular fashion, at the expense of file size, although this can be mitigated by compression or by using sparse arrays, which is a future goal. The change to netCDF4 is an important improvement for compatibility of result files across future versions of DeLTA.

Conclusion

DeLTA 3.0 enhances function for both novice and experienced users with new features (including command-line interface, lineage tree manipulation and growth rate calculation), better interoperability (results stored as netCDF files), and backend-agnostic deep-learning models (implemented in pure keras).

Acknowledgements

The authors acknowledge the contributions of Idris Kempf, Caroline Blassick, and Eric Bueno to the development of version 3.0 of DeLTA; feedback from the rest of the Dunlop lab; and input from the wider community through interactions on DeLTA's gitlab repository.

This work was supported by the National Science Foundation (MCB-2143289) to MJD. VA acknowledges support from the European Research Council (ERC-2022-STG, BridgingScales, grant agreement 101075989).

As of this version, direct dependencies of DeLTA include numpy (Harris et al., 2020), scipy (Virtanen et al., 2020), scikit-image (Walt et al., 2014), xarray (Hoyer & Hamman, 2017), pooch (Uieda et al., 2020), opencv (Bradski, 2000), tqdm (Tqdm/Tqdm, 2025), ffmpeg (Tomar, 2006), netCDF4 (Rew et al., 1989), bioio (Brown et al., 2023), termcolor (Termcolor/Termcolor, 2025), keras (Chollet & Others, 2015), matplotlib (Hunter, 2007), and elasticdeform (Tulder & Fringeli, 2024).

Utilities that we use in the development process include ruff (Astral-Sh/Ruff, 2025), mypy (Python/Mypy, 2025), deptry (Maas, 2025), pixi (Arts et al., 2025), pip (Pypa/Pip, 2025), pytest (Krekel et al., 2004), pytest-cov (Pytest-Dev/Pytest-Cov, 2025), pre-commit (Pre-Commit/Pre-Commit, 2025), sphinx (Sphinx-Doc/Sphinx, 2025), numpydoc (Numpy/Numpydoc, 2025), sphinx-design (Executablebooks/Sphinx-Design, 2025), and furo (Gedam, 2025).

AI Usage Disclosure

No generative AI was used in the software creation, documentation, or paper authoring.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jozefowicz, R., Jia, Y., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). *TensorFlow, Large-scale machine learning on heterogeneous systems*. <https://doi.org/10.5281/zenodo.4724125>
- Ahmadi, A., Courtney, M., Ren, C., & Ingalls, B. (2024). A benchmarked comparison of software packages for time-lapse image processing of monolayer bacterial population dynamics. *Microbiology Spectrum*. <https://doi.org/10.1128/spectrum.00032-24>
- Allard, P., Papazotos, F., & Potvin-Trottier, L. (2022). Microfluidics for long-term single-cell time-lapse microscopy: Advances and applications. *Frontiers in Bioengineering and Biotechnology*, 10. <https://doi.org/10.3389/fbioe.2022.968342>
- Ansel, J., Yang, E., He, H., Gimpelstein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., ... Chintala, S. (2024, April). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. <https://doi.org/10.1145/3620665.3640366>
- Arts, R., Zalmstra, B., Vollprecht, W., Jager, T. de, Morcotilo, N., & Hofer, J. (2025). *Pixi*. <https://github.com/prefix-dev/pixi/releases/tag/v0.53.0>
- Astral-sh/ruff. (2025). Astral. <https://github.com/astral-sh/ruff>
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). <http://github.com/google/jax>
- Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.
- Brown, E. M., Toloudis, D., Sherman, J., Swain-Bowden, M., Lambert, T., Meharry, S., Whitney, B., & BioIO Contributors. (2023). *BioIO: Image reading, metadata conversion, and image writing for microscopy images in pure python*. GitHub. <https://github.com/bioio-devs/bioio>
- Chollet, F., & Others. (2015). *Keras*. <https://keras.io>
- Cutler, K. J., Stringer, C., Lo, T. W., Rappez, L., Stroustrup, N., Brook Peterson, S., Wiggins, P. A., & Mougous, J. D. (2022). Omnipose: A high-precision morphology-independent solution for bacterial cell segmentation. *Nature Methods*, 19(11), 1438–1448. <https://doi.org/10.1038/s41592-022-01639-4>
- Executablebooks/sphinx-design. (2025). Executable Books. <https://github.com/executablebooks/sphinx-design>
- Gallusser, B., & Weigert, M. (2025). TRACKASTRA: Transformer-based cell tracking for live-cell microscopy. In A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, & G. Varol (Eds.), *Computer vision – ECCV 2024* (Vol. 15134, pp. 467–484). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-73116-7_27
- Gedam, P. (2025). *Pradyunsg/furo*. <https://github.com/pradyunsg/furo>
- Gericke, B., Degner, F., Hüttmann, T., Werth, S., & Fortmann-Grote, C. (2025). *Performance review of retraining and transfer learning of DeLTA2 for image segmentation for pseudomonas fluorescens SBW25*. 273–280. ISBN: 978-989-758-688-0
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,

- Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hoyer, S., & Hamman, J. (2017). Xarray: N-d labeled arrays and datasets in python. *Journal of Open Research Software*, 5(1). <https://doi.org/10.5334/jors.148>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Krekel, H., Oliveira, B., Pfannschmidt, R., Bruynooghe, F., Laughner, B., & Bruhin, F. (2004). *Pytest 7.5 (Version 7.5)*. <https://github.com/pytest-dev/pytest>
- Lugagne, J.-B., Lin, H., & Dunlop, M. J. (2020). DeLTA: Automated cell segmentation, tracking, and lineage reconstruction using deep learning. *PLOS Computational Biology*, 16(4), e1007673. <https://doi.org/10.1371/journal.pcbi.1007673>
- Maas, F. (2025). *Fpgmaas/deptr*. <https://github.com/fpgmaas/deptr>
- Numpy/numpydoc*. (2025). NumPy. <https://github.com/numpy/numpydoc>
- O'Connor, O. M., Alnahhas, R. N., Lugagne, J.-B., & Dunlop, M. J. (2022). DeLTA 2.0: A deep learning pipeline for quantifying single-cell spatial and temporal dynamics. *PLOS Computational Biology*, 18(1), e1009797. <https://doi.org/10.1371/journal.pcbi.1009797>
- O'Connor, O. M., & Dunlop, M. J. (2025). Cell-TRACTR: A transformer-based model for end-to-end segmentation and tracking of cells. *PLOS Computational Biology*, 21(5), e1013071. <https://doi.org/10.1371/journal.pcbi.1013071>
- Perez, F., Granger, B. E., & Hunter, J. D. (2011). Python: An ecosystem for scientific computing. *Computing in Science & Engineering*, 13(2), 13–21. <https://doi.org/10.1109/MCSE.2010.119>
- Pre-commit/pre-commit*. (2025). pre-commit. <https://github.com/pre-commit/pre-commit>
- Pypa/pip*. (2025). Python Packaging Authority. <https://github.com/pypa/pip>
- Pytest-dev/pytest-cov*. (2025). pytest-dev. <https://github.com/pytest-dev/pytest-cov>
- Python/mypy*. (2025). Python. <https://github.com/python/mypy>
- Rew, R., Davis, G., Emmerson, S., Cormack, C., Caron, J., Pincus, R., Hartnett, E., Heimbigner, D., Appel, L., & Fisher, W. (1989). *Unidata NetCDF*. UCAR/NCAR - Unidata. <https://doi.org/10.5065/D6H70CW6>
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In N. Navab, J. Hornegger, W. M. Wells, & A. F. Frangi (Eds.), *Medical image computing and computer-assisted intervention – MICCAI 2015* (Vol. 9351, pp. 234–241). Springer International Publishing. https://doi.org/10.1007/978-3-319-24574-4_28
- Savitzky, Abraham., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. <https://doi.org/10.1021/ac60214a047>
- Schmidt, U., Weigert, M., Broaddus, C., & Myers, G. (2018). Cell detection with star-convex polygons. In A. F. Frangi, J. A. Schnabel, C. Davatzikos, C. Alberola-López, & G. Fichtinger (Eds.), *Medical image computing and computer assisted intervention – MICCAI 2018* (Vol. 11071, pp. 265–273). Springer International Publishing. https://doi.org/10.1007/978-3-030-00934-2_30
- Sphinx-doc/sphinx*. (2025). Sphinx. <https://github.com/sphinx-doc/sphinx>

- 249 Stewart, E. J., Madden, R., Paul, G., & Taddei, F. (2005). Aging and death in an organism
250 that reproduces by morphologically symmetric division. *PLOS Biology*, 3(2), e45. <https://doi.org/10.1371/journal.pbio.0030045>
251
- 252 Stirling, D. R., Swain-Bowden, M. J., Lucas, A. M., Carpenter, A. E., Cimini, B. A., &
253 Goodman, A. (2021). CellProfiler 4: Improvements in speed, utility and usability. *BMC*
254 *Bioinformatics*, 22(1), 1–11. <https://doi.org/10.1186/s12859-021-04344-9>
- 255 Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: A generalist
256 algorithm for cellular segmentation. *Nature Methods*, 18(1), 100–106. <https://doi.org/10.1038/s41592-020-01018-x>
257
- 258 *Termcolor/termcolor*. (2025). termcolor. <https://github.com/termcolor/termcolor>
- 259 The HDF Group. (2025). *Hierarchical data format, version 5*. <https://github.com/HDFGroup/hdf5>
260
- 261 Thiermann, R., Sandler, M., Ahir, G., Sauls, J. T., Schroeder, J., Brown, S., Le Treut, G.,
262 Si, F., Li, D., Wang, J. D., & Jun, S. (2024). Tools and methods for high-throughput
263 single-cell imaging with the mother machine. *eLife*, 12, RP88463. <https://doi.org/10.7554/eLife.88463>
264
- 265 Tomar, S. (2006). Converting video formats with FFmpeg. *Linux J*, 2006(146), 10.
- 266 *Tqdm/tqdm*. (2025). tqdm developers. <https://github.com/tqdm/tqdm>
- 267 Tulder, G. van, & Fringeli, G. (2024). *Gvtulder/elasticdeform: Version v0.5.1*. Zenodo.
268 <https://doi.org/10.5281/zenodo.11267397>
- 269 Uieda, L., Soler, S. R., Rampin, R., Kemenade, H. van, Turk, M., Shapero, D., Banihirwe, A.,
270 & Leeman, J. (2020). Pooch: A friend to fetch your data files. *Journal of Open Source*
271 *Software*, 5(45), 1943. <https://doi.org/10.21105/joss.01943>
- 272 Valen, D. A. V., Kudo, T., Lane, K. M., Macklin, D. N., Quach, N. T., DeFelice, M.
273 M., Maayan, I., Tanouchi, Y., Ashley, E. A., & Covert, M. W. (2016). Deep learning
274 automates the quantitative analysis of individual cells in live-cell imaging experiments. *PLOS*
275 *Computational Biology*, 12(11), e1005177. <https://doi.org/10.1371/journal.pcbi.1005177>
- 276 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
277 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,
278 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E.,
279 ... Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
280 in python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- 281 Walt, S. van der, Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager,
282 N., Gouillart, E., & Yu, T. (2014). Scikit-image: Image processing in python. *PeerJ*, 2,
283 e453. <https://doi.org/10.7717/peerj.453>
- 284 Wang, P., Robert, L., Pelletier, J., Dang, W. L., Taddei, F., Wright, A., & Jun, S. (2010).
285 Robust growth of escherichia coli. *Current Biology*, 20(12), 1099–1103. <https://doi.org/10.1016/j.cub.2010.04.045>
286
- 287 Weigert, M., Schmidt, U., Haase, R., Sugawara, K., & Myers, G. (2020). Star-convex
288 polyhedra for 3D object detection and segmentation in microscopy. *2020 IEEE Winter*
289 *Conference on Applications of Computer Vision (WACV)*, 3655–3662. <https://doi.org/10.1109/WACV45572.2020.9093435>
290